حلقه‌ها و تکرار عملیات در پایتون

در برنامه‌نویسی، گاهی لازم است یک عملیات را چندین بار انجام دهیم. برای مثال، می‌خواهیم نام همه دانشجویان را نمایش دهیم، قیمت چند محصول را جمع کنیم یا از کاربر بخواهیم تا زمانی که اطلاعات معتبر وارد نکرده است، دوباره تلاش کند.

اگر بخواهیم برای هر بار تکرار، دستورهای مشابهی بنویسیم، کد طولانی و نگهداری آن دشوار می‌شود. در پایتون، با استفاده از حلقه‌ها (Loops) می‌توانیم مجموعه‌ای از دستورها را چند بار اجرا کنیم.

دو حلقه پرکاربرد پایتون عبارت‌اند از:

- `for` برای تکرار عملیات روی مجموعه‌ای از داده‌ها یا تعداد مشخصی از مراحل.
- `while` برای تکرار عملیات تا زمانی که یک شرط برقرار باشد.

۱. حلقه `for`

حلقه `for` زمانی کاربرد دارد که بخواهیم روی عناصر یک مجموعه حرکت کنیم یا عملیاتی را برای تعداد مشخصی از موارد انجام دهیم.

برای مثال، فرض کنید فهرستی از نام‌ها داریم و می‌خواهیم نام هر شخص را نمایش دهیم:

```
students = ["Sara", "Ali", "Reza"]

for student in students:
    print(student)
```

خروجی:

```
Sara
Ali
Reza
```

در این مثال، متغیر `student` در هر مرحله یکی از عناصر لیست را دریافت می‌کند و دستور `print()` برای آن اجرا می‌شود.

حلقه پس از بررسی همه عناصر لیست به پایان می‌رسد.

تکرار با `range()`

گاهی می‌خواهیم یک دستور را تعداد مشخصی بار اجرا کنیم، بدون آنکه از قبل یک لیست داشته باشیم. در این حالت می‌توانیم از `range()` استفاده کنیم.

```
for number in range(5):
    print(number)
```

خروجی:

```
0
1
2
3
4
```

عبارت `range(5)` اعدادی از صفر تا چهار را تولید می‌کند. عدد پایانی، یعنی پنج، در این محدوده قرار نمی‌گیرد.

بنابراین، `range(5)` پنج بار تکرار ایجاد می‌کند.

می‌توانیم نقطه شروع و پایان را نیز مشخص کنیم:

```
for number in range(1, 6):
    print(number)
```

خروجی:

```
1
2
3
4
5
```

در این مثال، تکرار از یک شروع می‌شود و تا پیش از شش ادامه می‌یابد.

همچنین می‌توانیم فاصله بین اعداد را تعیین کنیم:

```
for number in range(0, 11, 2):
    print(number)
```

خروجی:

```
0
2
4
6
8
10
```

در اینجا، عدد سوم در `range(0, 11, 2)` مشخص می‌کند که هر بار دو واحد به عدد قبلی اضافه شود.

یک کاربرد عملی

فرض کنید می‌خواهیم مجموع اعداد یک تا پنج را محاسبه کنیم:

```
total = 0

for number in range(1, 6):
    total = total + number

print(total)
```

خروجی:

```
15
```

در هر تکرار، مقدار متغیر `number` به `total` اضافه می‌شود. پس از پایان حلقه، متغیر `total` مجموع اعداد را نگه می‌دارد.

می‌توانیم عبارت جمع را کوتاه‌تر نیز بنویسیم:

```
total += number
```

این عبارت معادل `total = total + number` است.

۲. پردازش داده‌ها با حلقه for

یکی از کاربردهای مهم حلقه‌ها، انجام عملیات روی داده‌های ذخیره‌شده در ساختارهایی مانند لیست است.

برای مثال، فرض کنید قیمت چند محصول را در یک لیست قرار داده‌ایم و می‌خواهیم مجموع قیمت‌ها را محاسبه کنیم:

```
prices = [120, 250, 80, 150]

total = 0

for price in prices:
    total += price

print(f"Total price: {total}")
```

خروجی:

```
Total price: 600
```

در این برنامه، به‌جای نوشتن یک دستور جمع برای هر محصول، از یک حلقه استفاده کرده‌ایم. اگر تعداد محصولات بیشتر شود، نیازی به تغییر ساختار حلقه نداریم؛ فقط کافی است داده‌های جدید را به لیست اضافه کنیم.

این ویژگی در برنامه‌های واقعی اهمیت زیادی دارد. برای مثال، ممکن است بخواهیم اطلاعات صدها محصول، کاربر یا تراکنش را پردازش کنیم.

۳. حلقه `while`

حلقه `while` تا زمانی اجرا می‌شود که شرط تعیین‌شده درست باشد.

برای مثال:

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

خروجی:

```
1
2
3
4
5
```

در این برنامه، مراحل اجرای حلقه به این صورت است:

1. مقدار اولیه `count` برابر یک است.
2. شرط `count <= 5` بررسی می‌شود.
3. اگر شرط درست باشد، مقدار `count` نمایش داده می‌شود.
4. سپس یک واحد به `count` اضافه می‌شود.
5. اجرای حلقه با بررسی دوباره شرط ادامه پیدا می‌کند.

وقتی `count` به شش برسد، شرط نادرست می‌شود و حلقه پایان می‌یابد.

اهمیت تغییر وضعیت در حلقه

در حلقه `while` باید توجه کنیم که چه اتفاقی باعث نادرست شدن شرط می‌شود.

برای مثال، کد زیر مشکل دارد:

```
count = 1

while count <= 5:
    print(count)
```

در این برنامه، مقدار `count` تغییر نمی‌کند؛ بنابراین شرط همیشه درست باقی می‌ماند و حلقه به‌طور نامحدود اجرا می‌شود.

به چنین وضعیتی حلقه بی‌نهایت (Infinite Loop) می‌گوییم.

برای جلوگیری از آن، باید مطمئن شویم که در شرایط مناسب، وضعیت برنامه تغییر می‌کند یا راه دیگری برای پایان دادن به حلقه وجود دارد.

۴. دریافت ورودی تا زمانی که شرط برقرار است

گاهی نمی‌دانیم کاربر چند بار تلاش خواهد کرد. برای مثال، می‌خواهیم تا زمانی که کاربر رمز مشخصی را وارد نکرده است، از او ورودی بگیریم.

```
password = ""

while password != "python123":
    password = input("Enter the password: ")

print("Access granted.")
```

در این مثال، برنامه تا زمانی که مقدار ورودی برابر با رمز تعیین‌شده نباشد، از کاربر دوباره ورودی می‌گیرد.

وقتی ورودی با مقدار مورد انتظار برابر شود، شرط نادرست خواهد شد و حلقه پایان می‌یابد.

این صرفاً یک مثال آموزشی است؛ در برنامه‌های واقعی، ذخیره و بررسی رمز عبور به روش‌های امن‌تری نیاز دارد.

۵. تفاوت `for` و `while`

هر دو حلقه برای تکرار عملیات استفاده می‌شوند، اما معمولاً در موقعیت‌های متفاوتی کاربرد دارند.

ویژگی	`for`	`while`
کاربرد رایج	پیمایش داده‌ها یا تکرار در محدوده مشخص	تکرار تا زمانی که شرطی برقرار باشد
مثال	نمایش تمام عناصر یک لیست	دریافت ورودی تا رسیدن به مقدار معتبر
پایان حلقه	معمولاً پس از پایان پیمایش	زمانی که شرط نادرست شود
نکته مهم	انتخاب مجموعه یا محدوده مناسب	توجه به تغییر وضعیت و جلوگیری از حلقه بی‌نهایت

برای مثال، وقتی لیستی از محصولات داریم و می‌خواهیم قیمت همه آن‌ها را بررسی کنیم، `for` معمولاً انتخاب مناسبی است.

اما وقتی می‌خواهیم از کاربر ورودی بگیریم تا زمانی که شرط خاصی برقرار شود، `while` می‌تواند مناسب‌تر باشد.

البته در برخی مسئله‌ها می‌توان از هر دو حلقه استفاده کرد. انتخاب بهتر به خوانایی کد و ماهیت مسئله بستگی دارد.

۶. پایان زودهنگام با `break`

گاهی می‌خواهیم حلقه را پیش از رسیدن به پایان طبیعی آن متوقف کنیم. دستور `break` برای این کار استفاده می‌شود.

برای مثال، فرض کنید می‌خواهیم در یک فهرست به‌دنبال عدد مشخصی بگردیم:

```
numbers = [4, 8, 12, 16, 20]

for number in numbers:
    if number == 12:
        print("Number found.")
        break

    print(number)
```

خروجی:

```
4
8
Number found.
```

وقتی مقدار 12 پیدا می‌شود، دستور `break` حلقه را متوقف می‌کند. بنابراین، اعداد بعد از آن بررسی نمی‌شوند.

این دستور زمانی مفید است که ادامه تکرار دیگر ضرورتی نداشته باشد.

۷. رد کردن یک مرحله با `continue`

برخلاف `break` که کل حلقه را متوقف می‌کند، دستور `continue` فقط ادامه دستورهای همان مرحله را رد می‌کند و اجرای حلقه را از مرحله بعد ادامه می‌دهد.

برای مثال، می‌خواهیم اعداد یک تا پنج را نمایش دهیم، اما عدد سه را نادیده بگیریم:

```
for number in range(1, 6):
    if number == 3:
        continue

    print(number)
```

خروجی:

```
1
2
4
5
```

وقتی مقدار `number` برابر سه می‌شود، دستور `continue` باعث می‌شود `()print` در آن مرحله اجرا نشود. سپس حلقه به سراغ عدد بعدی می‌رود.

در نتیجه:

- `break` حلقه را پایان می‌دهد.
- `continue` مرحله فعلی را رد می‌کند و تکرار را ادامه می‌دهد.

۸. استفاده از شرط داخل حلقه

حلقه‌ها و شرط‌ها را می‌توانیم با هم ترکیب کنیم تا تنها داده‌هایی را پردازش کنیم که معیار مشخصی دارند.

برای مثال، می‌خواهیم از میان چند نمره، فقط نمره‌های قبولی را نمایش دهیم:

```
scores = [12, 8, 19, 15, 6, 10]

for score in scores:
    if score >= 10:
        print(score)
```

خروجی:

```
12
19
15
10
```

در این برنامه، حلقه روی تمام نمره‌ها حرکت می‌کند و شرط `if` تعیین می‌کند کدام نمره نمایش داده شود.

این ترکیب در بسیاری از کارها کاربرد دارد؛ مانند فیلتر کردن محصولات بر اساس قیمت، پیدا کردن کاربران واجد شرایط یا بررسی نتایج یک محاسبه.

۹. حلقه‌های تو در تو

گاهی برای حل یک مسئله به بیش از یک حلقه نیاز داریم. اگر یک حلقه درون حلقه دیگری قرار بگیرد، آن‌ها را حلقه‌های تو در تو (Nested Loops) می‌نامیم.

برای مثال:

```
for row in range(1, 4):
    for column in range(1, 3):
        print(row, column)
```

خروجی:

```
1 1
1 2
2 1
2 2
3 1
3 2
```

در این مثال، برای هر مقدار `row`، حلقه داخلی تمام مقادیر `column` را پیمایش می‌کند.

حلقه‌های تو در تو برای کار با جدول‌ها، داده‌های دوبعدی و برخی محاسبات کاربرد دارند. با این حال، هر حلقه اضافی می‌تواند تعداد عملیات را افزایش دهد؛ بنابراین بهتر است فقط زمانی از آن‌ها استفاده کنیم که ساختار مسئله به آن نیاز دارد.

جمع‌بندی

حلقه‌ها به ما امکان می‌دهند یک عملیات را بدون تکرار دستی دستورها، بارها اجرا کنیم.

حلقه `for` معمولاً برای پیمایش مجموعه‌ای از داده‌ها یا تکرار در یک محدوده مشخص مناسب است. حلقه `while` برای موقعیت‌هایی کاربرد دارد که تکرار باید تا زمان برقرار بودن یک شرط ادامه پیدا کند.

دستورهای `break` و `continue` نیز به ما کمک می‌کنند جریان اجرای حلقه را کنترل کنیم.

تمرین

برنامه‌ای بنویسید که قیمت چند محصول را در یک لیست نگهداری کند و مجموع قیمت‌ها را محاسبه کند.

برای مثال:

```
prices = [120, 250, 80, 150]
```

برنامه باید:

1. با استفاده از حلقه `for` روی قیمت‌ها حرکت کند.
2. مجموع قیمت‌ها را محاسبه کند.
3. تعداد محصولات را نمایش دهد.
4. میانگین قیمت محصولات را محاسبه و نمایش دهد.

تمرین تکمیلی: برنامه‌ای بنویسید که با استفاده از حلقه `while` از کاربر عدد دریافت کند و اعداد واردشده را جمع بزند. با وارد کردن عدد `0`، دریافت ورودی متوقف شود و مجموع نهایی نمایش داده شود.